SQL Interview Questions with Answers (Freshers - Data Science / ML)

1. Basics

1. What is SQL? Difference between SQL and NoSQL?

- SQL is Structured Query Language used for relational databases. NoSQL databases are non-relational, schema-less, and handle unstructured data.

2. Explain the difference between INNER JOIN, LEFT JOIN, RIGHT JOIN, and FULL JOIN.

- INNER JOIN: returns matching rows in both tables.
- LEFT JOIN: returns all rows from left table + matches from right.
- RIGHT JOIN: returns all rows from right table + matches from left.
- FULL JOIN: returns all rows when there is a match in one of the tables.

3. Difference between WHERE and HAVING?

- WHERE filters rows before aggregation. HAVING filters after aggregation.

4. Remove duplicates from a table?

SELECT DISTINCT column_name FROM table;

5. Difference between DELETE, TRUNCATE, DROP?

- DELETE: removes rows, can have WHERE clause.
- TRUNCATE: removes all rows, faster, no rollback.
- DROP: removes entire table.

6. PRIMARY KEY vs UNIQUE KEY?

- PRIMARY KEY: unique + not null.
- UNIQUE KEY: unique but can have nulls.

7. What is a foreign key?

- A constraint linking one table's column to another table's primary key.

8. Select distinct values?

SELECT DISTINCT column_name FROM table;

9. CHAR vs VARCHAR?

- CHAR: fixed length. VARCHAR: variable length.

10. Normalization?

- Process of organizing data to reduce redundancy.

2. Query Writing

11. Second highest salary:

SELECT MAX(salary) FROM employees WHERE salary < (SELECT MAX(salary) FROM employees);

12. Count employees per department:

SELECT department_id, COUNT(*) FROM employees GROUP BY department_id;

13. Customers with >5 orders:

SELECT customer_id FROM orders GROUP BY customer_id HAVING COUNT(*) > 5;

14. Employees joined last 30 days:

SELECT * FROM employees WHERE join_date >= CURRENT_DATE - INTERVAL '30 days';

15. Average order value per customer:

SELECT customer_id, AVG(order_amount) FROM orders GROUP BY customer_id;

16. Employees without managers:

SELECT * FROM employees WHERE manager_id IS NULL;

17. Top 3 products by sales:

SELECT product_id, SUM(sales) FROM orders GROUP BY product_id ORDER BY SUM(sales) DESC LIMIT 3;

18. Users signed up but never purchased:

SELECT u.user_id FROM users u LEFT JOIN orders o ON u.user_id = o.user_id WHERE o.user_id IS NULL;

19. Total revenue per month:

SELECT DATE_TRUNC('month', order_date) AS month, SUM(order_amount) FROM orders GROUP BY month;

20. Duplicate emails:

SELECT email, COUNT(*) FROM users GROUP BY email HAVING COUNT(*) > 1;

3. Aggregations & Window Functions

21. GROUP BY vs PARTITION BY:

- GROUP BY aggregates rows. PARTITION BY divides rows into groups for window functions.

22. Running totals:

SELECT order_id, SUM(order_amount) OVER (ORDER BY order_date) AS running_total FROM orders;

23. First and last purchase date per customer:

SELECT customer_id, MIN(order_date), MAX(order_date) FROM orders GROUP BY customer_id;

24. 7-day moving average:

SELECT order_date, AVG(sales) OVER (ORDER BY order_date ROWS BETWEEN 6 PRECEDING AND CURRENT ROW) AS moving_avg FROM sales;

25. Rank employees by salary per department:

SELECT employee_id, department_id, RANK() OVER (PARTITION BY department_id ORDER BY salary DESC) FROM employees;

26. Retention (consecutive months):

- Use window functions to check if user appears in consecutive months.

27. Percentage contribution:

SELECT product_id, SUM(sales)/SUM(SUM(sales)) OVER() * 100 AS pct_contribution FROM orders GROUP BY product_id;

28. Median salary:

SELECT PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY salary) FROM employees;

29. Churn rate:

- Count users inactive for 30+ days.

30. Cumulative revenue growth:

SELECT order_date, SUM(order_amount) OVER (ORDER BY order_date) AS cumulative_revenue FROM orders;

4. Data Cleaning & Transformation

31. Handle NULLs: Use COALESCE() or ISNULL().

32. Replace missing ages with median: Use UPDATE with subquery for median.

33. Lowercase names: SELECT LOWER(name) FROM customers;

34. Extract domain from email: SELECT SUBSTRING(email FROM POSITION('@' IN email)+1);

35. Calculate age: SELECT EXTRACT(YEAR FROM AGE(dob));

36. Trim spaces: SELECT TRIM(name) FROM customers;

37. Categorize spenders: Use CASE WHEN logic.

38. Pivot sales data: Use CASE WHEN with SUM().

39. Unpivot data: Use UNION ALL.

40. Fraud vs non-fraud counts: SELECT is_fraud, COUNT(*) FROM transactions GROUP BY is_fraud;

5. Case Studies (ML-Oriented)

41. Features per user: total spend, avg spend, frequency using GROUP BY.

42. Class imbalance: count fraud vs non-fraud.

43. Join demographics + transactions: JOIN on user_id.

44. Session length: difference between max and min timestamp per session.

45. Outliers: use HAVING amount > AVG(amount)+3*STDDEV(amount).

46. Conversion rate: signups vs purchases ratio.

47. CLV: sum of all purchases per user.

48. Repeat purchase rate: users with count(order_id) > 1.

49. Avg time between purchases: use LAG(order_date) window function.

50. Recommendation dataset: join user, product, and transaction tables.